

Day 06: Booking Service & Seat Management

By the end of this session, learners will be able to:

- Design theatre and screen data models
- Implement seat layout and selection systems
- Develop show scheduling mechanisms
- Understand and implement end-to-end booking workflows
- Handle seat availability and locking logic
- Analyze and solve concurrent booking problems
- Prevent double booking scenarios
- Manage booking lifecycle states

Service Overview: Booking Service

The Booking Service is responsible for:

- Managing theatre and screen configurations
- Scheduling movie shows
- Handling seat selection and availability
- Processing bookings
- Ensuring data consistency under concurrency

Academic Insight:

Booking systems are examples of high-consistency, transaction-sensitive systems, where incorrect design can lead to data anomalies such as double booking.

Theatre & Screen Design

Conceptual Entities

- Theatre
- Screen (Audi)
- Location

Example Structure

```
{  
  "theatreId": "T1",  
  "name": "PVR Cinemas",  
  "location": "Delhi",  
  "screens": ["Screen1", "Screen2"]  
}
```

Screen (Audi) Design

```
{  
  "screenId": "S1",  
  "name": "Audi 1",  
  "capacity": 100  
}
```

Design Considerations

- One theatre → multiple screens
- Each screen → independent seat layout
- Location drives availability

Insight:

Proper hierarchy design ensures data organization and efficient querying.

Seat Layout Modeling

Seat Representation: Seats are modeled as a 2D grid.

Example Layout

A1	A2	A3	A4
B1	B2	B3	B4
C1	C2	C3	C4

Seat Attributes

```
{  
  "seatId": "A1",  
  "type": "REGULAR",  
  "price": 200  
}
```

Design Approaches

- Static layout (predefined)
- Dynamic layout (configurable)

Academic Insight:

Seat modeling requires balancing simplicity vs flexibility, especially when pricing tiers are involved.

Show Scheduling

A show represents:

- A movie
- A screen
- A specific time

Example

```
{  
  "showId": "SH1",  
  "movieId": "M1",  
  "screenId": "S1",  
  "startTime": "10:00 AM"  
}
```

Constraints

- No overlapping shows on same screen
- Time slots must be validated

Insight: Scheduling problems involve time conflict resolution, a classic systems design challenge.

Booking Workflow

End-to-End Flow

- User selects movie
- Selects theatre & show
- Views seat layout
- Selects seats
- Booking request initiated
- Seats locked
- Payment (simulated)
- Booking confirmed

Workflow Diagram (Conceptual)

User → Select Seats → Lock Seats → Confirm Booking → Update Status

Academic Insight:

This flow represents a transactional workflow, where intermediate failures must be handled carefully.

Seat Availability Logic

Seat States

- Available
- Locked (temporary)
- Booked

Logic

- If seat is booked → unavailable
- If seat is locked → temporarily unavailable

Example

```
{  
  "seatId": "A1",  
  "status": "BOOKED"  
}
```

Insight:

Seat availability must be real-time and consistent across users.

Concurrent Booking Problem

Problem Statement: Multiple users try to book the same seat at the same time.

Example Scenario

- User A selects A1
- User B selects A1 simultaneously
- Both attempt booking

Result Without Control

- Double booking occurs

Academic Insight:

This is a classic race condition problem in distributed systems.

Preventing Double Booking

Approaches

1. Pessimistic Locking
 - Lock seat immediately
 - Others cannot access
2. Optimistic Locking
 - Check before update
 - Fail if already booked
3. Temporary Seat Locking (Recommended)
 - Lock seat for a limited time (e.g., 5 minutes)
 - Release if not booked

Conceptual Flow

Select Seat → Lock → Timer → Confirm or Release

Insight:

Temporary locking provides a balance between user experience and system consistency.

Booking Status Flow

Booking States

State	Description
INITIATED	Booking started

LOCKED	Seats reserved
CONFIRMED	Booking successful
CANCELLED	Booking cancelled
EXPIRED	Lock timeout

Example

```
{
  "bookingId": "B1",
  "status": "CONFIRMED"
}
```

Academic Insight:

State transitions follow a finite state machine (FSM) model, ensuring controlled workflow progression.

Data Consistency Considerations

Challenges

- Concurrent updates
- Distributed transactions
- Partial failures

Solutions

- Transaction management
- Locking strategies
- Idempotent APIs

Insight:

Consistency is critical in booking systems, often prioritized over availability (CAP theorem trade-off).

API Design (Conceptual)

Core APIs

Operation	Endpoint	Method
Fetch theatres	/theatres	GET
Fetch shows	/shows	GET
Seat Layout	/seats/{showId}	GET

Create booking	/booking	POST
----------------	----------	------

Integration with Other Services

- Movie Service → fetch movie details
- Auth Service → validate user
- API Gateway → route requests

Expected Learning Outcomes

Students will be able to:

- Design theatre and seat data models
- Implement booking workflows
- Handle concurrency in distributed systems
- Prevent double booking
- Manage booking lifecycle

Deliverables

- Theatre & screen models implemented
- Seat layout system completed
- Show scheduling implemented
- Booking workflow functional
- Seat locking logic implemented
- Double booking prevention mechanism
- Booking status flow managed

Viva Questions

1. What is a booking system and why is it complex?
2. Explain theatre and screen hierarchy design.
3. How is seat layout modeled in systems?
4. What is show scheduling?
5. Explain the booking workflow.
6. What is seat availability logic?
7. What is a race condition?
8. Explain concurrent booking problem.
9. What is double booking and how can it be prevented?
10. Differentiate between pessimistic and optimistic locking.
11. What is temporary seat locking?
12. Explain booking status lifecycle.
13. What is data consistency in distributed systems?
14. How does concurrency affect system design?

15. What is CAP theorem in context of booking systems?
16. What are challenges in real-time seat booking?
17. How do microservices coordinate during booking?
18. Why is transaction management important?